

The Design Philosophy of the DARPA Internet Protocols

DARPA 互联网协议的设计哲学

David D. Clark

Massachusetts Institute of Technology · Laboratory for Computer Science · Cambridge, MA 02139

大卫·D·克拉克 | 麻省理工学院 · 计算机科学实验室 · 马萨诸塞州剑桥市，邮编 02139

(Originally published in Proc. SIGCOMM '88, Computer Communication Review Vol. 18, No. 4, August 1988, pp. 106-114)

(原载于 SIGCOMM '88 会议论文集, 《计算机通信评论》第 18 卷第 4 期, 1988 年 8 月, 第 106-114 页)

* This work was supported in part by the Defense Advanced Research Projects Agency (DARPA) under Contract No. N00014-83-K-0125.

* 本研究部分由美国国防高级研究计划局 (DARPA) 资助, 合同编号 N00014-83-K-0125。

译者说明: 本文档为 David D. Clark 《The Design Philosophy of the DARPA Internet Protocols》(1988) 的中英对照全文翻译。每段先列英文原文 (灰色小字), 其后为中文译文 (黑色)。应要求采用严格直译, 尽量贴合原文的措辞与句式, 故个别中文表述会略显生硬。参考文献按学术惯例保留英文原文。

Abstract 摘要

The Internet protocol suite, TCP/IP, was first proposed fifteen years ago. It was developed by the Defense Advanced Research Projects Agency (DARPA), and has been used widely in military and commercial systems. While there have been papers and specifications that describe how the protocols work, it is sometimes difficult to deduce from these why the protocol is as it is. For example, the Internet protocol is based on a connectionless or datagram mode of service. The motivation for this has been greatly misunderstood. This paper attempts to capture some of the early reasoning which shaped the Internet protocols.

互联网协议族 TCP/IP 是在十五年前首次提出的。它由美国国防高级研究计划局 (DARPA) 开发, 已在军用和商用系统中得到广泛使用。尽管已有一些论文和规范描述了这些协议如何工作, 但有时却很难从中推断出协议为什么会是现在这个样子。例如, 互联网协议基于一种无连接 (或称数据报) 的服务模式, 而这样设计的动机一直被严重误解。本文试图捕捉那些塑造了互联网协议的早期推理中的一部分。

1. Introduction 1. 引言

For the last 15 years, the Advanced Research Projects Agency of the U.S. Department of Defense has been developing a suite of protocols for packet switched networking. These protocols, which include the Internet Protocol (IP), and the Transmission Control Protocol (TCP), are now U.S. Department of Defense standards for internetworking, and are in wide use in the commercial networking environment. The ideas developed in this effort have also influenced other protocol suites, most importantly the connectionless configuration of the ISO protocols.

在过去的 15 年里, 美国国防部高级研究计划局一直在开发一套用于分组交换网络的协议。这些协议包括互联网协议 (IP) 和传输控制协议 (TCP), 如今已成为美国国防部的网络互联标准, 并在商用网络环境中得到广泛使用。在这项工作中形成的思想也影响了其他协议族, 其中最重要的是 ISO 协议的无连接配置。

While specific information on the DOD protocols is fairly generally available, it is sometimes difficult to determine the motivation and reasoning which led to the design.

尽管关于国防部（DOD）这些协议的具体信息相当容易获得，但有时却很难确定促成该设计的动机与推理。

In fact, the design philosophy has evolved considerably from the first proposal to the current standards. For example, the idea of the datagram, or connectionless service, does not receive particular emphasis in the first paper, but has come to be the defining characteristic of the protocol. Another example is the layering of the architecture into the IP and TCP layers. This seems basic to the design, but was also not a part of the original proposal. These changes in the Internet design arose through the repeated pattern of implementation and testing that occurred before the standards were set.

事实上，从最初的提案到当前的标准，这套设计哲学已经发生了相当大的演变。例如，数据报（即无连接服务）的思想在最早的论文中并未受到特别强调，但后来却成为了该协议的决定性特征。另一个例子是将体系结构分层为 IP 层与 TCP 层。这看似是设计的基础，但同样也不是最初提案的一部分。互联网设计中的这些变化，是通过在标准确定之前反复进行的实现与测试这一过程而产生的。

The Internet architecture is still evolving. Sometimes a new extension challenges one of the design principles, but in any case an understanding of the history of the design provides a necessary context for current design extensions. The connectionless configuration of ISO protocols has also been colored by the history of the Internet suite, so an understanding of the Internet design philosophy may be helpful to those working with ISO. This paper catalogs one view of the original objectives of the Internet architecture, and discusses the relation between these goals and the important features of the protocols.

互联网体系结构仍在不断演变。有时，一项新的扩展会挑战某条设计原则，但无论如何，对设计历史的理解都为当前的设计扩展提供了必要的背景。ISO 协议的无连接配置也受到了互联网协议族历史的影响，因此理解互联网的设计哲学对那些从事 ISO 工作的人可能会有帮助。本文记录了对互联网体系结构最初目标的一种看法，并讨论这些目标与协议重要特征之间的关系。

2. Fundamental Goal 2. 根本目标

The top level goal for the DARPA Internet Architecture was to develop an effective technique for multiplexed utilization of existing interconnected networks. Some elaboration is appropriate to make clear the meaning of that goal.

DARPA 互联网体系结构的顶层目标，是开发一种能够对已有的、相互连接的网络进行多路复用利用的有效技术。有必要做一些详细说明，以阐明这一目标的含义。

The components of the Internet were networks, which were to be interconnected to provide some larger service. The original goal was to connect together the original ARPANET with the ARPA packet radio network, in order to give users on the packet radio network access to the large service machines on the ARPANET. At the time it was assumed that there would be other sorts of networks to interconnect, although the local area network had not yet emerged.

互联网的组成部分是各个网络，它们将被互相连接起来，以提供某种更大的服务。最初的目标是将原有的 ARPANET 与 ARPA 分组无线电网连接在一起，以便让分组无线电网上的用户能

够访问 ARPANET 上的大型服务机器。在当时，人们假定还会有其他种类的网络需要互联，尽管那时局域网尚未出现。

An alternative to interconnecting existing networks would have been to design a unified system which incorporated a variety of different transmission media, a multi-media network. While this might have permitted a higher degree of integration, and thus better performance, it was felt that it was necessary to incorporate the then existing network architectures if Internet was to be useful in a practical sense. Further, networks represent administrative boundaries of control, and it was an ambition of this project to come to grips with the problem of integrating a number of separately administrated entities into a common utility.

与互联已有网络相对的另一种做法，本可以是设计一个统一的系统，把各种不同的传输介质纳入其中，即一个多介质网络。尽管这或许能带来更程度的集成、从而获得更好的性能，但人们认为，如果互联网要在实际意义上有用，就必须把当时已经存在的各种网络体系结构纳入进来。此外，网络代表着管理控制的边界，而本项目的一个抱负就是要解决如何将若干独立管理的实体整合为一个公共设施这一问题。

The technique selected for multiplexing was packet switching. An alternative such as circuit switching could have been considered, but the applications being supported, such as remote login, were naturally served by the packet switching paradigm, and the networks which were to be integrated together in this project were packet switching networks. So packet switching was accepted as a fundamental component of the Internet architecture.

所选用的多路复用技术是分组交换。本来也可以考虑诸如电路交换之类的替代方案，但所支持的应用（如远程登录）天然地适合分组交换这一范式，而本项目中要整合在一起的那些网络本身就是分组交换网络。因此，分组交换被接受为互联网体系结构的一个基本组成部分。

The final aspect of this fundamental goal was the assumption of the particular technique for interconnecting these networks. Since the technique of store and forward packet switching, as demonstrated in the previous DARPA project, the ARPANET, was well understood, the top level assumption was that networks would be interconnected by a layer of Internet packet switches, which were called gateways.

这一根本目标的最后一个方面，是对互联这些网络所采用的特定技术所作的假定。由于存储转发式分组交换技术（正如此前的 DARPA 项目 ARPANET 所演示的那样）已经得到充分理解，因此顶层的假定就是：各网络将通过一层互联网分组交换机来互相连接，这些交换机被称为网关。

From these assumptions comes the fundamental structure of the Internet: a packet switched communications facility in which a number of distinguishable networks are connected together using packet communications processors called gateways which implement a store and forward packet forwarding algorithm.

由这些假定便引出了互联网的基本结构：一个分组交换的通信设施，其中若干个可区分的网络通过称为网关的分组通信处理机连接在一起，这些网关实现了一种存储转发的分组转发算法。

3. Second Level Goals 3. 第二层级目标

The top level goal stated in the previous section contains the word "effective," without offering any definition of what an effective interconnection must achieve. The following list summarizes a more detailed set of goals which were established for the Internet architecture.

上一节所陈述的顶层目标中包含了“有效”一词，却没有给出有效的互联究竟必须达成什么的任何定义。下面这份清单概括了为互联网体系结构所确立的一组更为详细的目标。

1. Internet communication must continue despite loss of networks or gateways.
1. 即使在网络或网关发生损失的情况下，互联网通信也必须能够继续。
2. The Internet must support multiple types of communications service.
2. 互联网必须支持多种类型的通信服务。
3. The Internet architecture must accommodate a variety of networks.
3. 互联网体系结构必须能够容纳多种多样的网络。
4. The Internet architecture must permit distributed management of its resources.
4. 互联网体系结构必须允许对其资源进行分布式管理。
5. The Internet architecture must be cost effective.
5. 互联网体系结构必须具有成本效益。
6. The Internet architecture must permit host attachment with a low level of effort.
6. 互联网体系结构必须允许以较低的工作量将主机接入。
7. The resources used in the internet architecture must be accountable.
7. 互联网体系结构中所使用的资源必须是可计量（可计费）的。

This set of goals might seem to be nothing more than a checklist of all the desirable network features. It is important to understand that these goals are in order of importance, and an entirely different network architecture would result if the order were changed. For example, since this network was designed to operate in a military context, which implied the possibility of a hostile environment, survivability was put as a first goal, and accountability as a last goal. During wartime, one is less concerned with detailed accounting of resources used than with mustering whatever resources are available and rapidly deploying them in an operational manner. While the architects of the Internet were mindful of accountability, the problem received very little attention during the early stages of the design, and is only now being considered. An architecture primarily for commercial deployment would clearly place these goals at the opposite end of the list.

这一组目标看上去也许只不过是一份罗列了所有理想网络特性的清单。重要的是要理解，这些目标是按重要性排序的，如果改变这个顺序，就会产生一种完全不同的网络体系结构。例如，由于这个网络是为在军事环境下运行而设计的——这意味着存在敌对环境的可能性——所以存活性被列为第一目标，而可计量性被列为最后一个目标。在战时，人们对所用资源的详细计量并不那么在意，而更关心的是调集一切可用的资源、并以可作战的方式迅速部署它们。尽管互联网的设计

者们也注意到了可计量性，但在设计的早期阶段，这个问题几乎没有受到关注，直到现在才开始被考虑。一个主要面向商业部署的体系结构，显然会把这些目标放到清单上完全相反的位置。

Similarly, the goal that the architecture be cost effective is clearly on the list, but below certain other goals, such as distributed management, or support of a wide variety of networks. Other protocol suites, including some of the more popular commercial architectures, have been optimized to a particular kind of network, for example a long haul store and forward network built of medium speed telephone lines, and deliver a very cost effective solution in this context, in exchange for dealing somewhat poorly with other kinds of nets, such as local area nets.

同样地，“体系结构应具有成本效益”这一目标也明确地在清单之中，但它排在某些其他目标（如分布式管理、或对多种多样网络的支持）之下。其他一些协议族，包括一些较为流行的商用体系结构，已经针对某一特定种类的网络做了优化——例如一种由中速电话线路构成的长途存储转发网络——并在这种情形下提供了非常具有成本效益的解决方案，但代价是对其他种类的网络（如局域网）处理得相当糟糕。

The reader should consider carefully the above list of goals, and recognize that this is not a "motherhood" list, but a set of priorities which strongly colored the design decisions within the Internet architecture. The following sections discuss the relationship between this list and the features of the Internet.

读者应当仔细思考上面这份目标清单，并认识到：这并不是一份“放之四海而皆准、人人都会赞同的空泛”清单，而是一组强烈地影响了互联网体系结构内各项设计决策的优先级排序。以下各节将讨论这份清单与互联网各项特征之间的关系。

4. Survivability in the Face of Failure 4. 面对故障时的存活性

The most important goal on the list is that the Internet should continue to supply communications service, even though networks and gateways are failing. In particular, this goal was interpreted to mean that if two entities are communicating over the Internet, and some failure causes the Internet to be temporarily disrupted and reconfigured to reconstitute the service, then the entities communicating should be able to continue without having to reestablish or reset the high level state of their conversation. More concretely, at the service interface of the transport layer, this architecture provides no facility to communicate to the client of the transport service that the synchronization between the sender and the receiver may have been lost. It was an assumption in this architecture that synchronization would never be lost unless there was no physical path over which any sort of communication could be achieved. In other words, at the top of transport, there is only one failure, and it is total partition. The architecture was to mask completely any transient failure.

清单上最重要的目标是：即使在网络和网关发生故障的情况下，互联网也应当继续提供通信服务。具体而言，这一目标被解释为：如果有两个实体正在通过互联网进行通信，而某次故障导致互联网暂时中断、并经过重新配置以重建服务，那么这两个进行通信的实体应当能够继续通信，而无需重新建立或重置它们对话的高层状态。更具体地说，在传输层的服务接口处，本体系结构并不提供任何机制，来向传输服务的客户告知发送方与接收方之间的同步可能已经丢失。本体系结构中的一个假定是：除非根本不存在任何物理通路可供进行任何形式的通信，否则同步永远不会丢失。换句话说，在传输层的顶端，只存在一种故障，那就是彻底的分割（断裂）。本体系结构要做到完全屏蔽任何瞬态故障。

To achieve this goal, the state information which describes the on-going conversation must be protected. Specific examples of state information would be the number of packets transmitted, the number of packets acknowledged, or the number of outstanding flow control permissions. If the lower layers of the architecture lose this information, they will not be able to tell if data has been lost, and the application layer will have to cope with the loss of synchrony. This architecture insisted that this disruption not occur, which meant that the state information must be protected from loss.

为实现这一目标，描述正在进行的对话的状态信息必须受到保护。状态信息的具体例子包括：已传输的分组数量、已被确认的分组数量、或尚未用掉的流量控制许可的数量。如果体系结构的较低层丢失了这些信息，它们就无法判断数据是否已经丢失，应用层将不得不去应对同步的丧失。本体系结构坚持要求这种中断不得发生，这意味着状态信息必须受到保护、不被丢失。

In some network architectures, this state is stored in the intermediate packet switching nodes of the network. In this case, to protect the information from loss, it must be replicated. Because of the distributed nature of the replication, algorithms to ensure robust replication are themselves difficult to build, and few networks with distributed state information provide any sort of protection against failure. The alternative, which this architecture chose, is to take this information and gather it at the endpoint of the net, at the entity which is utilizing the service of the network. I call this approach to reliability "fate-sharing." The fate-sharing model suggests that it is acceptable to lose the state information associated with an entity if, at the same time, the entity itself is lost. Specifically, information about transport level synchronization is stored in the host which is attached to the net and using its communication service.

在某些网络体系结构中，这种状态被存储在网络的中间分组交换节点里。在这种情形下，为防止信息丢失，就必须对它进行复制（冗余备份）。由于这种复制具有分布式的性质，用以确保可靠复制的算法本身就很难构建，因而很少有具备分布式状态信息的网络能提供任何形式的故障防护。本体系结构所选择的另一种做法，是把这些信息收集到网络的端点处，即收集到正在使用该网络服务的那个实体身上。我把这种实现可靠性的方法称为“命运共担”（fate-sharing）。命运共担模型意味着：如果在丢失与某个实体相关联的状态信息的同时，该实体本身也一并丢失了，那么这种丢失是可以接受的。具体而言，关于传输层同步的信息，被存储在那台接入网络、并使用其通信服务的主机之中。

There are two important advantages to fate-sharing over replication. First, fate-sharing protects against any number of intermediate failures, whereas replication can only protect against a certain number (less than the number of replicated copies). Second, fate-sharing is much easier to engineer than replication. 相比复制，命运共担有两个重要的优点。第一，命运共担可以抵御任意数量的中间故障，而复制只能抵御一定数量的故障（这个数量小于副本的数量）。第二，命运共担在工程实现上比复制要容易得多。

There are two consequences to the fate-sharing approach to survivability. First, the intermediate packet switching nodes, or gateways, must not have any essential state information about on-going connections. Instead, they are stateless packet switches, a class of network design sometimes called a "datagram" network. Secondly, rather more trust is placed in the host machine than in an architecture where the network ensures the reliable delivery of data. If the host resident algorithms that ensure the sequencing and acknowledgment of data fail, applications on that machine are prevented from operation.

采用命运共担这种方式来实现存活性，会带来两个后果。第一，中间的分组交换节点（即网关）必须不持有任何关于正在进行的连接的关键状态信息。相反，它们是无状态的分组交换机，这一

类网络设计有时被称为"数据报"网络。第二，相比那种由网络来确保数据可靠交付的体系结构，这里被赋予主机的信任要多得多。如果驻留在主机上、用于确保数据排序与确认的那些算法失效了，那么该主机上的应用就无法运行。

Despite the fact that survivability is the first goal in the list, it is still second to the top level goal of interconnection of existing networks. A more survivable technology might have resulted from a single multi-media network design. For example, the Internet makes very weak assumptions about the ability of a network to report that it has failed. Internet is thus forced to detect network failures using Internet level mechanisms, with the potential for a slower and less specific error detection.

尽管存活性是清单上的第一个目标，但它仍然次于互联已有网络这一顶层目标。一项存活能力更强的技术，本来或许可以由单一的多介质网络设计产生出来。例如，互联网对网络报告自身已发生故障的能力只做了非常微弱的假定。因此，互联网被迫使用互联网层面的机制来检测网络故障，这就可能导致错误检测更慢、也更不具体。

5. Types of Service 5. 服务类型

The second goal of the Internet architecture is that it should support, at the transport service level, a variety of types of service. Different types of service are distinguished by differing requirements for such things as speed, latency and reliability. The traditional type of service is the bi-directional reliable delivery of data. This service, which is sometimes called a "virtual circuit" service, is appropriate for such applications as remote login or file transfer. It was the first service provided in the Internet architecture, using the Transmission Control Protocol (TCP). It was early recognized that even this service had multiple variants, because remote login required a service with low delay in delivery, but low requirements for bandwidth, while file transfer was less concerned with delay, but very concerned with high throughput. TCP attempted to provide both these types of service.

互联网体系结构的第二个目标是：它应当在传输服务这一层级上支持多种类型的服务。不同类型的服务，是依据对速度、延迟、可靠性等方面的不同要求来加以区分的。传统的服务类型是数据的双向可靠交付。这种服务有时被称为"虚电路"服务，适用于远程登录或文件传输之类的应用。它是互联网体系结构中所提供的第一种服务，使用的是传输控制协议（TCP）。人们很早就认识到，即便是这一种服务也存在多种变体，因为远程登录要求一种交付延迟低、但对带宽要求也低的服务，而文件传输则不那么在意延迟，却非常在意高吞吐量。TCP 试图同时提供这两种类型的服务。

The initial concept of TCP was that it could be general enough to support any needed type of service. However, as the full range of needed services became clear, it seemed too difficult to build support for all of them into one protocol.

TCP 最初的设想是：它可以足够通用，从而支持任何所需类型的服务。然而，随着所需服务的全部范围逐渐变得清晰，要把对所有这些服务的支持都构建进同一个协议之中，似乎太过困难了。

The first example of a service outside the range of TCP was support for XNET, the cross-Internet debugger. TCP did not seem a suitable transport for XNET for several reasons. First, a debugger protocol should not be reliable. This conclusion may seem odd, but under conditions of stress or failure (which may be exactly when a debugger is needed) asking for reliable communications may prevent any communications at all. It is much better to build a service which can deal with whatever gets through, rather than insisting that every byte sent be delivered in order. Second, if TCP is general enough to deal

with a broad range of clients, it is presumably somewhat complex. Again, it seemed wrong to expect support for this complexity in a debugging environment, which may lack even basic services expected in an operating system (e.g. support for timers.) So XNET was designed to run directly on top of the datagram service provided by Internet.

第一个超出 TCP 适用范围的服务实例，是对跨互联网调试器 XNET 的支持。出于若干原因，TCP 似乎并不是适合 XNET 的传输协议。第一，调试器协议不应当是可靠的。这个结论也许看起来很怪，但在受压或故障的条件下（而这可能恰恰是需要调试器的时候），要求可靠通信反而可能导致根本无法进行任何通信。建立一种能够处理任何能够通过的数据的服务，要远好于坚持要求所发送的每一个字节都必须按序交付。第二，如果 TCP 通用到足以应对范围广泛的各类客户，那么它想必就有些复杂。同样地，期望在一个调试环境中支持这种复杂性，似乎也是不妥的，因为调试环境甚至可能连操作系统中所期望的基本服务（例如对定时器的支持）都缺乏。因此，XNET 被设计成直接运行在互联网所提供的数据报服务之上。

Another service which did not fit TCP was real time delivery of digitized speech, which was needed to support the teleconferencing aspect of command and control applications. In real time digital speech, the primary requirement is not a reliable service, but a service which minimizes and smoothes the delay in the delivery of packets. The application layer is digitizing the analog speech, packetizing the resulting bits, and sending them out across the network on a regular basis. They must arrive at the receiver at a regular basis in order to be converted back to the analog signal. If packets do not arrive when expected, it is impossible to reassemble the signal in real time. A surprising observation about the control of variation in delay is that the most serious source of delay in networks is the mechanism to provide reliable delivery. A typical reliable transport protocol responds to a missing packet by requesting a retransmission and delaying the delivery of any subsequent packets until the lost packet has been retransmitted. It then delivers that packet and all remaining ones in sequence. The delay while this occurs can be many times the round trip delivery time of the net, and may completely disrupt the speech reassembly algorithm. In contrast, it is very easy to cope with an occasional missing packet. The missing speech can simply be replaced by a short period of silence, which in most cases does not impair the intelligibility of the speech to the listening human. If it does, high level error correction can occur, and the listener can ask the speaker to repeat the damaged phrase.

另一种不适合 TCP 的服务，是数字化语音的实时传送，这是支持指挥与控制应用中电话会议（远程会议）这一方面所需要的。在实时数字语音中，首要的要求不是一种可靠的服务，而是一种能够最小化并平滑分组交付延迟的服务。应用层对模拟语音进行数字化，把得到的比特打包成分组，并按固定的节奏将它们发送到网络上。这些分组必须按固定的节奏到达接收端，才能被转换回模拟信号。如果分组没有在预期的时刻到达，就不可能实时地重组信号。关于对延迟变化进行控制，有一个令人惊讶的观察：网络中最严重的延迟来源，正是用来提供可靠交付的那套机制。一个典型的可靠传输协议在面对一个丢失的分组时，会通过请求重传来作出响应，并把任何后续分组的交付都推迟，直到那个丢失的分组被重传为止。然后它再按顺序交付那个分组以及其余所有分组。在此过程中产生的延迟可能是网络往返交付时间的许多倍，并且可能彻底打乱语音重组算法。与此相反，要应付偶尔丢失一个分组却非常容易。丢失的那段语音可以简单地用一小段静音来替代，在大多数情况下这并不会损害听者对语音的可理解性。如果确实有所损害，还可以进行高层的纠错，听者可以请说话者把受损的那句话重复一遍。

It was thus decided, fairly early in the development of the Internet architecture, that more than one transport service would be required, and the architecture must be prepared to tolerate simultaneously transports which wish to constrain reliability, delay, or bandwidth, at a minimum.

因此，在互联网体系结构开发的相当早期就已经决定：将需要不止一种传输服务，而且该体系结构至少必须准备好同时容纳那些希望在可靠性、延迟或带宽方面加以约束的各种传输。

This goal caused TCP and IP, which originally had been a single protocol in the architecture, to be separated into two layers. TCP provided one particular type of service, the reliable sequenced data stream, while IP attempted to provide a basic building block out of which a variety of types of service could be built. This building block was the datagram, which had also been adopted to support survivability. Since the reliability associated with the delivery of a datagram was not guaranteed, but "best effort," it was possible to build out of the datagram a service that was reliable (by acknowledging and retransmitting at a higher level), or a service which traded reliability for the primitive delay characteristics of the underlying network substrate. The User Datagram Protocol (UDP) was created to provide a application-level interface to the basic datagram service of Internet.

这一目标导致 TCP 与 IP——它们在体系结构中原本是单一的一个协议——被分离为两个层。TCP 提供一种特定类型的服务，即可靠的、按序的数据流，而 IP 则试图提供一种基本的构件，可以用它来构建出多种类型的服务。这个构件就是数据报，它同时也被采用来支持存活性。由于与数据报交付相关联的可靠性并不被保证，而只是“尽力而为”（best effort），所以既有可能用数据报构建出一种可靠的服务（通过在更高层进行确认与重传），也有可能构建出一种以可靠性为代价、换取底层网络基底原始延迟特性的服务。用户数据报协议（UDP）就是为给互联网的基本数据报服务提供一个应用层接口而创建的。

The architecture did not wish to assume that the underlying networks themselves support multiple types of services, because this would violate the goal of using existing networks. Instead, the hope was that multiple types of service could be constructed out of the basic datagram building block using algorithms within the host and the gateway. For example, (although this is not done in most current implementations) it is possible to take datagrams which are associated with a controlled delay but unreliable service and place them at the head of the transmission queues unless their lifetime has expired, in which case they would be discarded; while packets associated with reliable streams would be placed at the back of the queues, but never discarded, no matter how long they had been in the net.

该体系结构不希望假定底层网络自身就支持多种类型的服务，因为这样会违背使用已有网络这一目标。相反，人们所希望的是：能够利用主机和网关内部的算法，从基本的数据报构件出发，构造出多种类型的服务。例如（尽管在当前大多数实现中并未这样做），可以把那些与受控延迟但不可靠服务相关联的数据报放到传输队列的队首，除非它们的生存期已经到期——在那种情况下它们会被丢弃；而把那些与可靠数据流相关联的分组放到队列的队尾，但无论它们在网络中已经停留了多久，都绝不丢弃。

It proved more difficult than first hoped to provide multiple types of service without explicit support from the underlying networks. The most serious problem was that networks designed with one particular type of service in mind were not flexible enough to support other services. Most commonly, a network will have been designed under the assumption that it should deliver reliable service, and will inject delays as a part of producing reliable service, whether or not this reliability is desired. The interface behavior defined by X.25, for example, implies reliable delivery, and there is no way to turn this feature off. Therefore, although Internet operates successfully over X.25 networks it cannot deliver the desired variability of type

service in that context. Other networks which have an intrinsic datagram service are much more flexible in the type of service they will permit, but these networks are much less common, especially in the long-haul context.

事实证明，在得不到底层网络明确支持的情况下提供多种类型的服务，比最初所希望的要困难得多。最严重的问题在于：那些在设计时只考虑了某一种特定服务类型的网络，灵活性不足以支持其他服务。最常见的情形是，一个网络在设计时所基于的假定是它应当交付可靠的服务，于是它就会把延迟作为产生可靠服务的一部分注入进去，而不管这种可靠性是否被需要。例如，X.25 所定义的接口行为就隐含着可靠交付，而且没有办法把这个特性关掉。因此，尽管互联网能够成功地在 X.25 网络上运行，但在那种环境下它无法交付所期望的那种服务类型的可变性。另一些本身具有内在数据报服务的网络，在它们所允许的服务类型上要灵活得多，但这类网络要少见得多，尤其是在长途网络的环境中。

6. Varieties of Networks 6. 网络的多样性

It was very important for the success of the Internet architecture that it be able to incorporate and utilize a wide variety of network technologies, including military and commercial facilities. The Internet architecture has been very successful in meeting this goal; it is operated over a wide variety of networks, including long haul nets (the ARPANET itself and various X.25 networks), local area nets (Ethernet, ringnet, etc.), broadcast satellite nets (the DARPA Atlantic Satellite Network operating at 64 kilobits per second and the DARPA Experimental Wideband Satellite Net, operating within the United States at 3 megabits per second), packet radio networks (the DARPA packet radio network, as well as an experimental British packet radio net and a network developed by amateur radio operators), a variety of serial links, ranging from 1200 bit per second asynchronous connections to T1 links, and a variety of other ad hoc facilities, including intercomputer busses and the transport service provided by the higher layers of other network suites, such as IBM's HASP.

对互联网体系结构的成功而言，它能够纳入并利用各种各样的网络技术（包括军用和商用设施）这一点非常重要。互联网体系结构在实现这一目标上一直非常成功；它运行在各种各样的网络之上，包括长途网络（ARPANET 本身以及各种 X.25 网络）、局域网（以太网、环形网等）、广播卫星网络（以 64 千比特每秒运行的 DARPA 大西洋卫星网络，以及在美国境内以 3 兆比特每秒运行的 DARPA 实验性宽带卫星网络）、分组无线网络（DARPA 分组无线网络，以及一个实验性的英国分组无线网络和一个由业余无线电操作者开发的网络）、各种串行链路（从 1200 比特每秒的异步连接到 T1 链路）、以及各种其他临时性的设施，包括计算机间总线、以及由其他网络协议族的较高层所提供的传输服务（例如 IBM 的 HASP）。

The Internet architecture achieves this flexibility by making a minimum set of assumptions about the function which the net will provide. The basic assumption is that network can transport a packet or datagram. The packet must be of reasonable size, perhaps 100 bytes minimum, and should be delivered with reasonable but not perfect reliability. The network must have some suitable form of addressing if it is more than a point to point link.

互联网体系结构是通过网络将要提供的功能做出一组最小限度的假定，来实现这种灵活性的。最基本的假定是：网络能够传送一个分组或数据报。该分组的大小必须合理，也许最小为 100 字节，并且应当以合理但并非完美的可靠性被交付。如果该网络不只是一条点对点链路，那么它就必须具备某种合适的寻址形式。

There are a number of services which are explicitly not assumed from the network. These include reliable or sequenced delivery, network level broadcast or multicast, priority ranking of transmitted packet, support for multiple types of service, and internal knowledge of failures, speeds, or delays. If these services had been required, then in order to accommodate a network within the Internet, it would be necessary either that the network support these services directly, or that the network interface software provide enhancements to simulate these services at the endpoint of the network. It was felt that this was an undesirable approach, because these services would have to be re-engineered and reimplemented for every single network and every single host interface to every network. By engineering these services at the transport, for example reliable delivery via TCP, the engineering must be done only once, and the implementation must be done only once for each host. After that, the implementation of interface software for a new network is usually very simple.

有一些服务是明确地不从网络那里加以假定的。这些服务包括：可靠交付或按序交付、网络层级的广播或多播、对所传送分组的优先级排序、对多种服务类型的支持、以及对故障、速度或延迟的内部知情。如果这些服务被要求具备，那么为了把一个网络纳入互联网，就必须要么由该网络直接支持这些服务，要么由网络接口软件提供增强功能，以在网络的端点处模拟这些服务。人们觉得这是一种不可取的做法，因为这些服务将不得不为每一个网络、以及通往每个网络的每一个主机接口重新设计和重新实现。而通过把这些服务工程化地放在传输层来实现（例如经由 TCP 实现可靠交付），这项工程工作就只需做一次，对每台主机也只需实现一次。在此之后，为一个新网络实现接口软件通常就非常简单了。

7. Other Goals 7. 其他目标

The three goals discussed so far were those which had the most profound impact on the design on the architecture. The remaining goals, because they were lower in importance, were perhaps less effectively met, or not so completely engineered. The goal of permitting distributed management of the Internet has certainly been met in certain respects. For example, not all of the gateways in the Internet are implemented and managed by the same agency. There are several different management centers within the deployed Internet, each operating a subset of the gateways, and there is a two-tiered routing algorithm which permits gateways from different administrations to exchange routing tables, even though they do not completely trust each other, and a variety of private routing algorithms used among the gateways in a single administration. Similarly, the various organizations which manage the gateways are not necessarily the same organizations that manage the networks to which the gateways are attached.

到目前为止所讨论的三个目标，是对体系结构设计产生了最深远影响的那几个。其余的目标，由于其重要性较低，也许被满足得不那么有效，或者没有被那么完整地加以工程化。允许对互联网进行分布式管理这一目标，在某些方面无疑已经实现。例如，互联网中的网关并非全都由同一个机构来实现和管理。在已部署的互联网内部存在若干个不同的管理中心，每个中心运营着网关中的一个子集；而且存在一种两级的路由算法，它允许来自不同管理方的网关相互交换路由表，即便它们彼此之间并不完全信任；此外，在单个管理方内部的各网关之间，还使用着各种私有的路由算法。同样地，管理这些网关的各个组织，未必就是管理这些网关所接入的那些网络的组织。

On the other hand, some of the most significant problems with the Internet today relate to lack of sufficient tools for distributed management, especially in the area of routing. In the large internet being currently operated, routing decisions need to be constrained by policies for resource usage. Today this can be done only in a very limited way, which requires manual setting of tables. This is error-prone and at the same time not sufficiently powerful. The most important change in the Internet architecture over the

next few years will probably be the development of a new generation of tools for management of resources in the context of multiple administrations.

另一方面，如今互联网最重大的一些问题，与缺乏足够的分布式管理工具有关，尤其是在路由这一领域。在当前正在运营的大型互联网中，路由决策需要受到资源使用策略的约束。如今这一点只能以非常有限的方式能做到，需要手工设置路由表。这既容易出错，同时又不够强大。在未来几年里，互联网体系结构中最重要变化，很可能就是开发出新一代的工具，用于在多管理方并存的环境下进行资源管理。

It is clear that in certain circumstances, the Internet architecture does not produce as cost effective a utilization of expensive communication resources as a more tailored architecture would. The headers of Internet packets are fairly long (a typical header is 40 bytes), and if short packets are sent, this overhead is apparent. The worse case, of course, is the single character remote login packets, which carry 40 bytes of header and one byte of data. Actually, it is very difficult for any protocol suite to claim that these sorts of interchanges are carried out with reasonable efficiency. At the other extreme, large packets for file transfer, with perhaps 1,000 bytes of data, have an overhead for the header of only four percent.

很显然，在某些情形下，互联网体系结构对昂贵通信资源的利用，不如一种更为量身定制的体系结构那样具有成本效益。互联网分组的首部相当长（典型的首部为 40 字节），如果发送的是短分组，这种开销就很明显。当然，最糟糕的情况是单字符的远程登录分组，它携带着 40 字节的首部和 1 字节的数据。事实上，任何协议族都很难声称这类交换是以合理的效率进行的。而在另一个极端，用于文件传输的大分组（也许带有 1000 字节的数据），其首部的开销仅为百分之四。

Another possible source of inefficiency is retransmission of lost packets. Since Internet does not insist that lost packets be recovered at the network level, it may be necessary to retransmit a lost packet from one end of the Internet to the other. This means that the retransmitted packet may cross several intervening nets a second time, whereas recovery at the network level would not generate this repeat traffic. This is an example of the tradeoff resulting from the decision, discussed above, of providing services from the end-points. The network interface code is much simpler, but the overall efficiency is potentially less. However, if the retransmission rate is low enough (for example, 1%) then the incremental cost is tolerable. As a rough rule of thumb for networks incorporated into the architecture, a loss of one packet in a hundred is quite reasonable, but a loss of one packet in ten suggests that reliability enhancements be added to the network if that type of service is required.

另一个可能的低效来源是丢失分组的重传。由于互联网并不坚持要求在网络层级恢复丢失的分组，所以可能就需要把一个丢失的分组从互联网的一端重传到另一端。这意味着被重传的分组可能要第二次穿越若干个中间网络，而在网络层级进行恢复则不会产生这种重复的流量。这是上文所讨论的、由“从端点处提供服务”这一决策所导致的权衡的一个例子。网络接口代码要简单得多，但总体效率却可能更低。然而，如果重传率足够低（例如 1%），那么这种增量成本就是可以容忍的。作为对纳入本体系结构的各网络的一条粗略经验法则：每一百个分组丢失一个是相当合理的，但每十个分组就丢失一个则表明：如果需要那种类型的服务，就应当为该网络添加可靠性增强。

The cost of attaching a host to the Internet is perhaps somewhat higher than in other architectures, because all of the mechanisms to provide the desired types of service, such as acknowledgments and retransmission strategies, must be implemented in the host rather than in the network. Initially, to programmers who were not familiar with protocol implementation, the effort of doing this seemed somewhat daunting. Implementors tried such things as moving the transport protocols to a front end

processor, with the idea that the protocols would be implemented only once, rather than again for every type of host. However, this required the invention of a host to front end protocol which some thought almost as complicated to implement as the original transport protocol. As experience with protocols increases, the anxieties associated with implementing a protocol suite within the host seem to be decreasing, and implementations are now available for a wide variety of machines, including personal computers and other machines with very limited computing resources.

把一台主机接入互联网的成本，也许比在其他体系结构中要稍高一些，因为所有用于提供所期望服务类型的机制（例如确认和重传策略）都必须在主机中实现，而不是在网络中实现。起初，对那些不熟悉协议实现的程序员来说，做这件事所需的工作量显得有些令人望而生畏。实现者们尝试过诸如把传输协议迁移到一个前端处理机上之类的做法，其想法是协议只需实现一次，而不必为每一种类型的主机再实现一遍。然而，这就需要发明出一种主机到前端的协议，而有些人认为这种协议实现起来几乎和原本的传输协议一样复杂。随着关于协议的经验日益增多，与“在主机内部实现一套协议族”相关联的那些焦虑似乎正在减少，如今已经可以在各种各样的机器上获得相应的实现，包括个人计算机以及其他计算资源非常有限的机器。

A related problem arising from the use of host-resident mechanisms is that poor implementation of the mechanism may hurt the network as well as the host. This problem was tolerated, because the initial experiments involved a limited number of host implementations which could be controlled. However, as the use of Internet has grown, this problem has occasionally surfaced in a serious way. In this respect, the goal of robustness, which led to the method of fate-sharing, which led to host-resident algorithms, contributes to a loss of robustness if the host mis-behaves.

使用驻留于主机的机制还会引出一个相关的问题：对该机制的拙劣实现，可能既会伤害主机，也会伤害网络。这个问题在当初是被容忍的，因为最初的实验只涉及数量有限、且可以加以控制的主机实现。然而，随着互联网使用的增长，这个问题偶尔会以严重的方式浮现出来。在这一点上，对健壮性的追求导致了命运共担这一方法，而命运共担又导致了驻留于主机的算法，于是一旦主机行为不当，这一追求反而会造成健壮性的丧失。

The last goal was accountability. In fact, accounting was discussed in the first paper by Cerf and Kahn as an important function of the protocols and gateways. However, at the present time, the Internet architecture contains few tools for accounting for packet flows. This problem is only now being studied, as the scope of the architecture is being expanded to include non-military consumers who are seriously concerned with understanding and monitoring the usage of the resources within the internet.

最后一个目标是可计量性。事实上，在瑟夫（Cerf）和卡恩（Kahn）的第一篇论文中，就曾把计量作为协议和网关的一项重要功能加以讨论。然而，在目前，互联网体系结构中几乎没有用于对分组流进行计量的工具。这个问题直到现在开始被研究，因为该体系结构的适用范围正在扩大，开始把那些非军事的用户也纳入进来，而这些用户非常关心如何理解和监测互联网内部资源的使用情况。

8. Architecture and Implementation 8. 体系结构与实现

The previous discussion clearly suggests that one of the goals of the Internet architecture was to provide wide flexibility in the service offered. Different transport protocols could be used to provide different types of service, and different networks could be incorporated. Put another way, the architecture tried very hard not to constrain the range of service which the Internet could be engineered to provide. This, in turn,

means that to understand the service which can be offered by a particular implementation of an Internet, one must look not to the architecture, but to the actual engineering of the software within the particular hosts and gateways, and to the particular networks which have been incorporated. I will use the term "realization" to describe a particular set of networks, gateways and hosts which have been connected together in the context of the Internet architecture. Realizations can differ by orders of magnitude in the service which they offer. Realizations have been built out of 1200 bit per second phone lines, and out of networks only with speeds greater than 1 megabit per second. Clearly, the throughput expectations which one can have of these realizations differ by orders of magnitude. Similarly, some Internet realizations have delays measured in tens of milliseconds, where others have delays measured in seconds. Certain applications such as real time speech work fundamentally differently across these two realizations. Some Internets have been engineered so that there is great redundancy in the gateways and paths. These Internets are survivable, because resources exist which can be reconfigured after failure. Other Internet realizations, to reduce cost, have single points of connectivity through the realization, so that a failure may partition the Internet into two halves.

前面的讨论清楚地表明，互联网体系结构的目标之一，是在所提供的服务上具有广泛的灵活性。可以使用不同的传输协议来提供不同类型的服务，也可以把不同的网络纳入进来。换句话说，该体系结构竭力不去约束互联网经过工程实现后所能提供的服务范围。这反过来又意味着：要理解某个特定的互联网实现所能提供的服务，人们必须看的不是体系结构，而是特定主机和网关内部软件的实际工程实现，以及那些已被纳入进来的特定网络。我将使用“实现体”（realization）这个术语来描述一组特定的、在互联网体系结构的背景下被连接到一起的网络、网关和主机。各个实现体在它们所提供的服务上可以相差好几个数量级。有的实现体是用 1200 比特每秒的电话线路构建的，有的则只用速度大于 1 兆比特每秒的网络构建。显然，人们对这些实现体所能抱有的吞吐量预期会相差好几个数量级。同样地，有些互联网实现体的延迟以数十毫秒来衡量，而另一些延迟则以秒来衡量。某些应用（例如实时语音）在这两种实现体上的工作方式有着根本的不同。有些互联网在工程上经过设计，使得网关和路径都具有很大的冗余。这些互联网是可存活的，因为存在可以在故障之后重新配置的资源。另一些互联网实现体为了降低成本，在整个实现体中只设置了单一的连通点，因而一次故障就可能把互联网分割成两半。

The Internet architecture tolerates this variety of realization by design. However, it leaves the designer of a particular realization with a great deal of engineering to do. One of the major struggles of this architectural development was to understand how to give guidance to the designer of a realization, guidance which would relate the engineering of the realization to the types of service which would result. For example, the designer must answer the following sort of question. What sort of bandwidths must be in the underlying networks, if the overall service is to deliver a throughput of a certain rate? Given a certain model of possible failures within this realization, what sorts of redundancy ought to be engineered into the realization?

互联网体系结构在设计上就容忍这种多样化的实现体。然而，它也给特定实现体的设计者留下了大量的工程工作要做。这一体系结构开发过程中的一大难题，就是要弄清楚如何向实现体的设计者提供指导，这种指导要能够把实现体的工程实现与由此产生的服务类型联系起来。例如，设计者必须回答下面这一类问题：如果总体服务要交付某一速率的吞吐量，那么底层网络中必须具备何种带宽？在给定这个实现体内部某种可能故障模型的情况下，应当在实现体中工程化地设计进何种冗余？

Most of the known network design aids did not seem helpful in answering these sorts of questions. Protocol verifiers, for example, assist in confirming that protocols meet specifications. However, these tools almost never deal with performance issues, which are essential to the idea of the type of service. Instead, they deal with the much more restricted idea of logical correctness of the protocol with respect to specification. While tools to verify logical correctness are useful, both at the specification and implementation stage, they do not help with the severe problems that often arise related to performance. A typical implementation experience is that even after logical correctness has been demonstrated, design faults are discovered that may cause a performance degradation of an order of magnitude. Exploration of this problem has led to the conclusion that the difficulty usually arises, not in the protocol itself, but in the operating system on which the protocol runs. This being the case, it is difficult to address the problem within the context of the architectural specification. However, we still strongly feel the need to give the implementor guidance. We continue to struggle with this problem today.

大多数已知的网络设计辅助工具，在回答这类问题上似乎都帮不上忙。例如，协议验证器有助于确认协议符合规范。然而，这些工具几乎从不处理性能问题，而性能对于服务类型这一思想来说却是至关重要的。相反，它们所处理的是范围窄得多的概念，即协议相对于规范的逻辑正确性。验证逻辑正确性的工具虽然有用——无论是在规范阶段还是实现阶段——但它们对于那些常常与性能相关而出现的严重问题却毫无帮助。一个典型的实现经验是：即便在逻辑正确性已被证明之后，仍会发现一些设计上的缺陷，它们可能造成一个数量级的性能下降。对这一问题的探究得出了如下结论：困难通常并不出在协议本身，而是出在协议所运行其上的操作系统。既然如此，就很难在体系结构规范的范围之内去解决这个问题。然而，我们仍然强烈地感到有必要向实现者提供指导。直到今天，我们仍在为这个问题苦苦挣扎。

The other class of design aid is the simulator, which takes a particular realization and explores the service which it can deliver under a variety of loadings. No one has yet attempted to construct a simulator which take into account the wide variability of the gateway implementation, the host implementation, and the network performance which one sees within possible Internet realizations. It is thus the case that the analysis of most Internet realizations is done on the back of an envelope. It is a comment on the goal structure of the Internet architecture that a back of the envelope analysis, if done by a sufficiently knowledgeable person, is usually sufficient. The designer of a particular Internet realization is usually less concerned with obtaining the last five percent possible in line utilization than knowing whether the desired type of service can be achieved at all given the resources at hand at the moment.

另一类设计辅助工具是模拟器，它针对某个特定的实现体，探究该实现体在各种负载下所能交付的服务。然而，迄今还没有人尝试去构建一个模拟器，能够把人们在各种可能的互联网实现体中所见到的网关实现、主机实现以及网络性能的巨大可变性都考虑进去。于是情况就成了：对大多数互联网实现体的分析都是在信封背面（即粗略估算）上完成的。而一项信封背面式的分析，只要由一个足够有学识的人来做，通常就已经足够了——这正是对互联网体系结构目标构成的一种评说。某个特定互联网实现体的设计者，通常并不那么在意去争取线路利用率上最后那百分之五，而更在意的是：在当下手头资源的条件下，所期望的那种服务类型究竟能否实现。

The relationship between architecture and performance is an extremely challenging one. The designers of the Internet architecture felt very strongly that it was a serious mistake to attend only to logical correctness and ignore the issue of performance. However, they experienced great difficulty in formalizing any aspect of performance constraint within the architecture. These difficulties arose both because the goal of the architecture was not to constrain performance, but to permit variability, and secondly (and

perhaps more fundamentally), because there seemed to be no useful formal tools for describing performance.

体系结构与性能之间的关系是一个极富挑战性的问题。互联网体系结构的设计者们强烈地感到：只关注逻辑正确性而忽视性能问题，是一个严重的错误。然而，他们在把性能约束的任何方面在体系结构内部加以形式化时，都遇到了巨大的困难。这些困难的产生，一方面是因为该体系结构的目标并不是去约束性能，而是去容许可变性；另一方面（也许更为根本）是因为，似乎并不存在用以描述性能的有用的形式化工具。

This problem was particularly aggravating because the goal of the Internet project was to produce specification documents which were to become military standards. It is a well known problem with government contracting that one cannot expect a contractor to meet any criteria which is not a part of the procurement standard. If the Internet is concerned about performance, therefore, it was mandatory that performance requirements be put into the procurement specification. It was trivial to invent specifications which constrained the performance, for example to specify that the implementation must be capable of passing 1,000 packets a second. However, this sort of constraint could not be part of the architecture, and it was therefore up to the individual performing the procurement to recognize that these performance constraints must be added to the specification, and to specify them properly to achieve a realization which provides the required types of service. We do not have a good idea how to offer guidance in the architecture for the person performing this task.

这个问题尤其令人烦恼，因为互联网项目的目标是要产出将要成为军用标准的规范文档。政府承包中有一个众所周知的问题：你不能指望一个承包商去满足任何并不属于采购标准一部分的准则。因此，如果互联网在意性能，那就必须把性能要求写进采购规范之中。要发明出约束性能的规范是轻而易举的，例如规定该实现必须能够每秒通过 1000 个分组。然而，这一类约束却无法成为体系结构的一部分，因此就要靠执行采购的那个人去认识到：这些性能约束必须被添加到规范中，并要把它们恰当地规定下来，以获得一个能够提供所需服务类型的实现体。对于如何在体系结构中为执行这项任务的人提供指导，我们并没有什么好的想法。

9. Datagrams 9. 数据报

The fundamental architectural feature of the Internet is the use of datagrams as the entity which is transported across the underlying networks. As this paper has suggested, there are several reasons why datagrams are important within the architecture. First, they eliminate the need for connection state within the intermediate switching nodes, which means that the Internet can be reconstituted after a failure without concern about state. Secondly, the datagram provides a basic building block out of which a variety of types of service can be implemented. In contrast to the virtual circuit, which usually implies a fixed type of service, the datagram provides a more elemental service which the endpoints can combine as appropriate to build the type of service needed. Third, the datagram represents the minimum network service assumption, which has permitted a wide variety of networks to be incorporated into various Internet realizations. The decision to use the datagram was an extremely successful one, which allowed the Internet to meet its most important goals very successfully.

互联网最根本的体系结构特征，就是使用数据报来作为在底层网络上被传送的实体。正如本文所指出的，数据报在该体系结构中之所以重要，有几个原因。第一，它们消除了在中继交换节点内部保有连接状态的需要，这意味着互联网在一次故障之后可以被重建，而无需顾虑状态的问题。第二，数据报提供了一种基本的构件，可以用它来实现多种类型的服务。与通常隐含着固定服务

类型的虚电路相反，数据报提供的是一种更为基本的服务，端点可以视情况把它组合起来，以构建出所需要的服务类型。第三，数据报代表着最低限度的网络服务假定，正是这一点允许了各种各样的网络被纳入到各种互联网实现体之中。采用数据报的这个决定是一个极其成功的决定，它让互联网非常成功地实现了其最重要的目标。

There is a mistaken assumption often associated with datagrams, which is that the motivation for datagrams is the support of a higher level service which is essentially equivalent to the datagram. In other words, it has sometimes been suggested that the datagram is provided because the transport service which the application requires is a datagram service. In fact, this is seldom the case. While some applications in the Internet, such as simple queries of date servers or name servers, use an access method based on an unreliable datagram, most services within the Internet would like a more sophisticated transport model than simple datagram. Some services would like the reliability enhanced, some would like the delay smoothed and buffered, but almost all have some expectation more complex than a datagram. It is important to understand that the role of the datagram in this respect is as a building block, and not as a service in itself.

有一种常常与数据报联系在一起的错误假定，即认为数据报的动机是为了支持某种本质上与数据报等价的更高层服务。换句话说，有时有人提出：之所以提供数据报，是因为应用所要求的传输服务就是一种数据报服务。事实上，情况很少是这样。虽然互联网中的某些应用（例如对日期服务器或名字服务器的简单查询）使用了一种基于不可靠数据报的访问方法，但互联网中的大多数服务所希望的，是一种比简单数据报更为复杂精巧的传输模型。有些服务希望增强可靠性，有些希望对延迟加以平滑和缓冲，但几乎所有服务都有着某种比数据报更为复杂的期望。重要的是要理解：数据报在这方面所扮演的角色是一种构件，而不是其本身就是一种服务。

10. TCP 10. TCP

There were several interesting and controversial design decisions in the development of TCP, and TCP itself went through several major versions before it became a reasonably stable standard. Some of these design decisions, such as window management and the nature of the port address structure, are discussed in a series of implementation notes published as part of the TCP protocol handbook. But again the motivation for the decision is sometimes lacking. In this section, I attempt to capture some of the early reasoning that went into parts of TCP. This section is of necessity incomplete; a complete review of the history of TCP itself would require another paper of this length.

在 TCP 的开发过程中，有若干个有趣且具有争议的设计决策，而且 TCP 本身在成为一个相当稳定的标准之前，经历了好几个主要的版本。这些设计决策中的一部分（例如窗口管理以及端口地址结构的性质）在作为 TCP 协议手册一部分而发表的一系列实现注记中有所讨论。但同样地，做出这些决策的动机有时是缺失的。在本节中，我试图捕捉融入 TCP 某些部分之中的一些早期推理。本节注定是不完整的；要对 TCP 本身的历史做一次完整的回顾，将需要另写一篇这样篇幅的论文。

The original ARPANET host-to host protocol provided flow control based on both bytes and packets. This seemed overly complex, and the designers of TCP felt that only one form of regulation would be sufficient. The choice was to regulate the delivery of bytes, rather than packets. Flow control and acknowledgment in TCP is thus based on byte number rather than packet number. Indeed, in TCP there is no significance to the packetization of the data.

最初的 ARPANET 主机到主机协议同时基于字节和分组来提供流量控制。这看上去过于复杂，而 TCP 的设计者们觉得只用一种调控形式就足够了。所做出的选择是调控字节的交付，而不是分组的交付。因此，TCP 中的流量控制和确认是基于字节编号、而不是分组编号的。事实上，在 TCP 中，数据如何被分组（打包）并无任何意义。

This decision was motivated by several considerations, some of which became irrelevant and others of which were more important than anticipated. One reason to acknowledge bytes was to permit the insertion of control information into the sequence space of the bytes, so that control as well as data could be acknowledged. That use of the sequence space was dropped, in favor of ad hoc techniques for dealing with each control message. While the original idea has appealing generality, it caused complexity in practice.

这个决定是由若干考虑所驱动的，其中有些后来变得无关紧要，另一些则比预想的更为重要。确认字节的一个理由，是为了允许把控制信息插入到字节的序列空间之中，这样一来，控制信息和数据都可以被确认。对序列空间的这种用法后来被放弃了，转而采用临时性的技术来分别处理每一条控制消息。虽然最初的想法具有颇为吸引人的普遍性，但它在实践中造成了复杂性。

A second reason for the byte stream was to permit the TCP packet to be broken up into smaller packets if necessary in order to fit through a net with a small packet size. But this function was moved to the IP layer when IP was split from TCP, and IP was forced to invent a different method of fragmentation.

采用字节流的第二个理由，是为了在必要时允许把 TCP 分组拆分成更小的分组，以便能够通过一个分组尺寸较小的网络。但当 IP 从 TCP 中分离出来时，这一功能被移到了 IP 层，而 IP 则被迫发明出一种不同的分片方法。

A third reason for acknowledging bytes rather than packets was to permit a number of small packets to be gathered together into one larger packet in the sending host if retransmission of the data was necessary. It was not clear if this advantage would be important; it turned out to be critical. Systems such as UNIX which have an internal communication model based on single character interactions often send many packets with one byte of data in them. (One might argue from a network perspective that this behavior is silly, but it was a reality, and a necessity for interactive remote login.) It was often observed that such a host could produce a flood of packets with one byte of data, which would arrive much faster than a slow host could process them. The result is lost packets and retransmission.

确认字节而非分组的第三个理由，是为了在必须重传数据的情况下，允许发送主机把若干个小分组聚集到一个更大的分组之中。当时并不清楚这个优点是否会重要；结果证明它至关重要。诸如 UNIX 这样的系统，其内部通信模型是基于单字符交互的，常常会发送许多每个只带有一个字节数据的分组。（从网络的角度看，人们或许会说这种行为很愚蠢，但它确实是一个现实，而且对于交互式远程登录来说是一种必需。）人们经常观察到，这样的一台主机能够产生出大量每个只带一个字节数据的分组洪流，它们到达的速度远远快于一台慢速主机所能处理的速度。其结果就是分组丢失和重传。

If the retransmission was of the original packets, the same problem would repeat on every retransmission, with a performance impact so intolerable as to prevent operation. But since the bytes were gathered into one packet for retransmission, the retransmission occurred in a much more effective way which permitted practical operation.

如果重传的是原来那些（小）分组，那么同样的问题就会在每一次重传时重复出现，其对性能的影响之大，会令系统无法运行。但由于这些字节为了重传而被聚集到了一个分组之中，重传便以一种有效得多的方式发生，从而使实际运行成为可能。

On the other hand, the acknowledgment of bytes could be seen as creating this problem in the first place. If the basis of flow control had been packets rather than bytes, then this flood might never have occurred. Control at the packet level has the effect, however, of providing a severe limit on the throughput if small packets are sent. If the receiving host specifies a number of packets to receive, without any knowledge of the number of bytes in each, the actual amount of data received could vary by a factor of 1000, depending on whether the sending host puts one or one thousand bytes in each packet.

另一方面，确认字节这件事本身又可以被看作是一开始就制造了这个问题。如果流量控制的依据是分组而不是字节，那么这种洪流也许就根本不会发生。然而，在分组层级进行控制会带来这样一个后果：如果发送的是小分组，吞吐量就会受到严重的限制。如果接收主机指定它要接收若干个分组，却对每个分组中的字节数一无所知，那么实际接收到的数据量可能会相差 1000 倍，这取决于发送主机在每个分组中放入的是 1 个字节还是 1000 个字节。

In retrospect, the correct design decision may have been that if TCP is to provide effective support of a variety of services, both packets and bytes must be regulated, as was done in the original ARPANET protocols.

回过头来看，正确的设计决策也许应当是：如果 TCP 要对多种多样的服务提供有效的支持，那么就必须像最初的 ARPANET 协议所做的那样，对分组和字节都加以调控。

Another design decision related to the byte stream was the End-Of-Letter flag, or EOL. This has now vanished from the protocol, replaced by the Push flag, or PSH. The original idea of EOL was to break the byte stream into records. It was implemented by putting data from separate records into separate packets, which was not compatible with the idea of combining packets on retransmission. So the semantics of EOL was changed to a weaker form, meaning only that the data up to this point in the stream was one or more complete application-level elements, which should occasion a flush of any internal buffering in TCP or the network. By saying "one or more" rather than "exactly one", it became possible to combine several together and preserve the goal of compacting data in reassembly. But the weaker semantics meant that various applications had to invent an ad hoc mechanism for delimiting records on top of the data stream.

另一个与字节流相关的设计决策是“信件结束”标志，即 EOL（End-Of-Letter）。它如今已经从协议中消失，被推送标志（Push flag，即 PSH）所取代。EOL 最初的想法是把字节流切分成一条条记录。它的实现方式是来自不同记录的数据放进不同的分组里，而这与“在重传时合并分组”的想法并不兼容。于是 EOL 的语义被改成了一种较弱的形式，其含义仅仅是：数据流中到这一点为止的数据，构成了一个或多个完整的应用层单元，应当借此把 TCP 或网络中任何内部缓冲都冲刷（flush）出去。通过说“一个或多个”而不是“恰好一个”，就有可能把若干个合并到一起，从而保住在重组时压紧数据这一目标。但这种较弱的语义意味着，各种应用不得不在数据流之上发明出一种临时性的机制来界定记录的边界。

In this evolution of EOL semantics, there was a little known intermediate form, which generated great debate. Depending on the buffering strategy of the host, the byte stream model of TCP can cause great problems in one improbable case. Consider a host in which the incoming data is put in a sequence of

fixed size buffers. A buffer is returned to the user either when it is full, or an EOL is received. Now consider the case of the arrival of an out-of-order packet which is so far out of order to lie beyond the current buffer. Now further consider that after receiving this out-of-order packet, a packet with an EOL causes the current buffer to be returned to the user only partially full. This particular sequence of actions has the effect of causing the out of order data in the next buffer to be in the wrong place, because of the empty bytes in the buffer returned to the user. Coping with this generated book-keeping problems in the host which seemed unnecessary.

在 EOL 语义的这一演变过程中，曾有一种鲜为人知的中间形式，它引发了极大的争论。视主机的缓冲策略而定，TCP 的字节流模型在一种不大可能出现的情形下会造成很大的问题。设想这样一台主机：进入的数据被放入一连串固定大小的缓冲区中。一个缓冲区要么在它被填满时、要么在收到一个 EOL 时被交还给用户。现在考虑这样一种情况：到达了一个乱序分组，它乱序的程度之大，以至于落在了当前缓冲区之外。再进一步设想：在收到这个乱序分组之后，一个带有 EOL 的分组使得当前缓冲区在只被部分填满的情况下就被交还给用户。这一特定的动作序列所产生的后果是：由于被交还给用户的那个缓冲区中存在空字节，下一个缓冲区中的乱序数据就会处在错误的位置上。在主机中应付这种情况会产生一些看上去并无必要的记账（簿记）问题。

To cope with this it was proposed that the EOL should "use up" all the sequence space up to the next value which was zero mod the buffer size. In other words, it was proposed that EOL should be a tool for mapping the byte stream to the buffer management of the host.

为了应付这种情况，有人提议：EOL 应当“用掉”一直到下一个能被缓冲区大小整除（即对缓冲区大小取模为零）的数值为止的全部序列空间。换句话说，有人提议 EOL 应当成为一种把字节流映射到主机缓冲区管理的工具。

This idea was not well received at the time, as it seemed much too ad hoc, and only one host seemed to have this problem. In retrospect, it may have been the correct idea to incorporate into TCP some means of relating the sequence space and the buffer management algorithm of the host. At the time, the designers simply lacked the insight to see how that might be done in a sufficiently general manner.

这个想法在当时并不受欢迎，因为它显得太过临时凑就，而且似乎只有一台主机存在这个问题。回过头来看，把某种将序列空间与主机缓冲区管理算法关联起来的手段纳入 TCP，或许才是正确的想法。在当时，设计者们只是缺乏那种洞察力，看不出如何能以一种足够通用的方式来做这一点。

(Footnote) This use of EOL was properly called "Rubber EOL" but its detractors quickly called it "rubber baby buffer bumpers" in an attempt to ridicule the idea. Credit must go to the creator of the idea, Bill Plummer, for sticking to his guns in the face of detractors saying the above to him ten times fast.

（脚注）EOL 的这种用法本来被正式地称为“橡皮 EOL”（Rubber EOL），但它的反对者们很快就把它叫作“rubber baby buffer bumpers”（橡皮宝宝缓冲挡块——一句出了名难念的英语绕口令），企图以此嘲讽这个想法。必须把功劳归于这个想法的提出者比尔·普卢默（Bill Plummer）：在反对者们对着他飞快地连说十遍上面那句绕口令时，他依然坚持己见。

11. Conclusion 11. 结论

In the context of its priorities, the Internet architecture has been very successful. The protocols are widely used in the commercial and military environment, and have spawned a number of similar architectures. At the same time, its success has made clear that in certain situations, the priorities of the designers do not

match the needs of the actual users. More attention to such things as accounting, resource management and operation of regions with separate administrations are needed.

在其优先级的背景之下，互联网体系结构一直非常成功。这些协议在商用和军用环境中得到广泛使用，并催生出了若干个类似的体系结构。与此同时，它的成功也清楚地表明：在某些情形下，设计者们的优先级排序与实际用户的需求并不相符。需要对诸如计量、资源管理、以及对分属不同管理方的各区域的运营这类事情给予更多的关注。

While the datagram has served very well in solving the most important goals of the Internet, it has not served so well when we attempt to address some of the goals which were further down the priority list. For example, the goals of resource management and accountability have proved difficult to achieve in the context of datagrams. As the previous section discussed, most datagrams are a part of some sequence of packets from source to destination, rather than isolated units at the application level. However, the gateway cannot directly see the existence of this sequence, because it is forced to deal with each packet in isolation. Therefore, resource management decisions or accounting must be done on each packet separately. Imposing the datagram model on the internet layer has deprived that layer of an important source of information which it could use in achieving these goals.

虽然数据报在解决互联网最重要的目标方面表现得非常好，但当我们试图去处理优先级清单中位置更靠后的某些目标时，它的表现就不那么好了。例如，事实证明，资源管理和可计量性这两个目标在数据报的框架下很难实现。正如上一节所讨论的，大多数数据报都是从源到目的地的某个分组序列中的一部分，而不是应用层级上孤立的单元。然而，网关无法直接看到这个序列的存在，因为它被迫孤立地处理每一个分组。因此，资源管理决策或计量就必须对每一个分组分别进行。把数据报模型强加于互联网层之上，使该层被剥夺了一个本可用于实现这些目标的重要信息来源。

This suggests that there may be a better building block than the datagram for the next generation of architecture. The general characteristic of this building block is that it would identify a sequence of packets traveling from the source to the destination, without assuming any particular type of service with that service. I have used the word "flow" to characterize this building block. It would be necessary for the gateways to have flow state in order to remember the nature of the flows which are passing through them, but the state information would not be critical in maintaining the desired type of service associated with the flow. Instead, that type of service would be enforced by the end points, which would periodically send messages to ensure that the proper type of service was being associated with the flow. In this way, the state information associated with the flow could be lost in a crash without permanent disruption of the service features being used. I call this concept "soft state," and it may very well permit us to achieve our primary goals of survivability and flexibility, while at the same time doing a better job of dealing with the issue of resource management and accountability. Exploration of alternative building blocks constitute one of the current directions for research within the DARPA Internet Program.

这表明，对于下一代体系结构而言，也许存在一种比数据报更好的构件。这种构件的一般特征是：它将标识出一个从源到目的地传送的分组序列，而不为该序列假定任何特定类型的服务。我用“流”（flow）这个词来刻画这种构件。网关将有必要持有流状态，以便记住正穿过它们的那些流的性质，但这些状态信息对于维持与该流相关联的所期望服务类型而言并不是关键性的。相反，那种服务类型将由端点来强制保证，端点会周期性地发送消息，以确保与该流相关联的是恰当的服务类型。如此一来，与流相关联的状态信息即便在一次崩溃中丢失，也不会对正在使用的服务特性造成永久性的中断。我把这个概念称为“软状态”（soft state），它很可能让我们既能够实现

存活性和灵活性这两个首要目标，同时又能在处理资源管理和可计量性这一问题上做得更好。对各种替代性构件的探索，构成了 DARPA 互联网计划当前的研究方向之一。

12. Acknowledgments — A Historical Perspective 12. 致谢——一个历史的视角

It would be impossible to acknowledge all the contributors to the Internet project; there have literally been hundreds over the 15 years of development: designers, implementors, writers and critics. Indeed, an important topic, which probably deserves a paper in itself, is the process by which this project was managed. The participants came from universities, research laboratories and corporations, and they united (to some extent) to achieve this common goal.

要把互联网项目的所有贡献者都一一致谢是不可能的；在 15 年的开发历程中，贡献者实实在在地数以百计：设计者、实现者、撰写者和批评者。事实上，有一个重要的话题——它本身大概就值得专门写一篇论文——就是这个项目得以被管理的那一过程。参与者来自大学、研究实验室和企业，他们（在某种程度上）团结起来，去实现这一共同的目标。

The original vision for TCP came from Robert Kahn and Vinton Cerf, who saw very clearly, back in 1973, how a protocol with suitable features might be the glue that would pull together the various emerging network technologies. From their position at DARPA, they guided the project in its early days to the point where TCP and IP became standards for the DoD.

TCP 最初的构想来自罗伯特·卡恩（Robert Kahn）和文顿·瑟夫（Vinton Cerf），早在 1973 年，他们就非常清楚地看到：一个具备适当特性的协议，可以充当把各种正在兴起的网络技术黏合到一起的“胶水”。他们凭借在 DARPA 的位置，在项目早期对其加以引导，直至 TCP 和 IP 成为美国国防部（DoD）的标准。

The author of this paper joined the project in the mid-70s, and took over architectural responsibility for TCP/IP in 1981. He would like to thank all those who have worked with him, and particularly those who took the time to reconstruct some of the lost history in this paper.

本文作者于 20 世纪 70 年代中期加入该项目，并于 1981 年接管了 TCP/IP 的体系结构责任。他想要感谢所有曾与他共事的人，尤其是那些花费时间来为本文重建一部分已经失落的历史的人。

References 参考文献

（以下文献条目按学术惯例保留英文原文，未作翻译。）

1. V. Cerf, and R. Kahn, "A Protocol for Packet Network Intercommunication", IEEE Transactions on Communications, Vol. 22, No. 5, May 1974, pp. 637-648.
2. ISO, "Transport Protocol Specification", Tech. report IS-8073, International Organization for Standardization, September 1984.
3. ISO, "Protocol for Providing the Connectionless-Mode Network Service", Tech. report DIS8473, International Organization for Standardization, 1986.
4. R. Callon, "Internetwork Protocol", Proceedings of the IEEE, Vol. 71, No. 12, December 1983, pp. 1388-1392.
5. Jonathan B. Postel, "Internetwork Protocol Approaches", IEEE Transactions on Communications, Vol. Com-28, No. 4, April 1980, pp. 605-611.
6. Jonathan B. Postel, Carl A. Sunshine, Danny Cohen, "The ARPA Internet Protocol", Computer Networks 5, Vol. 5, No. 4, July 1981, pp. 261-271.

7. Alan Sheltzer, Robert Hinden, and Mike Brescia, "Connecting Different Types of Networks with Gateways", Data Communications, August 1982.
8. J. McQuillan and D. Walden, "The ARPA Network Design Decisions", Computer Networks, Vol. 1, No. 5, August 1977, pp. 243-289.
9. R. E. Kahn, S. A. Gronemeyer, J. Burdifiel, E. V. Hoversten, "Advances in Packet Radio Technology", Proceedings of the IEEE, Vol. 66, No. 11, November 1978, pp. 1408-1496.
10. B. M. Leiner, D. L. Nelson, F. A. Tobagi, "Issues in Packet Radio Design", Proceedings of the IEEE, Vol. 75, No. 1, January 1987, pp. 6-20.
11. "Transmission Control Protocol RFC-793", DDN Protocol Handbook, Vol. 2, September 1981, pp. 2.179-2.198.
12. Jack Haverty, "XNET Formats for Internet Protocol Version 4 IEN 158", DDN Protocol Handbook, Vol. 2, October 1980, pp. 2-345 to 2-348.
13. Jonathan Postel, "User Datagram Protocol NIC-RFC-768", DDN Protocol Handbook, Vol. 1, August 1980, pp. 2.175-2.177.
14. I. Jacobs, R. Binder, and E. Hoversten, "General Purpose Packet Satellite Networks", Proceedings of the IEEE, Vol. 66, No. 11, November 1978, pp. 1448-1467.
15. C. Topolcic and J. Kaiser, "The SATNET Monitoring System", Proceedings of the IEEE MILCOM, Boston, MA, October 1985, pp. 26.1.1-26.1.9.
16. W. Edmond, S. Blumenthal, A. Echenique, S. Storch, T. Calderwood, and T. Rees, "The Butterfly Satellite IMP for the Wideband Packet Satellite Network", Proceedings of the ACM SIGCOMM '86, ACM, Stowe, Vt., August 1986, pp. 194-203.
17. David D. Clark, "Window and Acknowledgment Strategy in TCP NIC-RFC-813", DDN Protocol Handbook, Vol. 3, July 1982, pp. 3-5 to 3-26.
18. David D. Clark, "Name, Addresses, Ports, and Routes NIC-RFC-814", DDN Protocol Handbook, Vol. 3, July 1982, pp. 3-27 to 3-40.